Redux Toolkit - Complete Documentation (Hinglish)

Contents

Redux Toolkit Documentation (Hinglish Mein)

1. Introduction

Redux Toolkit (RTK) Redux ka official aur recommended tarika hai state management karne ka. Purane Redux mein bahut zyada **boilerplate code** likhna padta tha (actions, action types, reducers sab alag-alag), lekin Redux Toolkit ye sab simplify kar deta hai.

Redux Toolkit kyun use karte hain?

- Kam boilerplate code
- `createSlice` se actions aur reducers ek jagah likh sakte hain
- Immutable updates automatically handle hoti hain (Immer library ke through)
- `configureStore` se store setup easy ho jata hai
- Async logic ke liye `createAsyncThunk` built-in milta hai
- DevTools automatically configure ho jaate hain

2. Installation

```
npm install @reduxjs/toolkit react-redux
```

`@reduxjs/toolkit` main package hai aur `react-redux` React components ko Redux store se connect karne ke liye chahiye.

3. Store Setup - configureStore

Sabse pehle ek store banate hain jisme saari application ka state rahega.

```
// app/store.js
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../features/counter/counterSlice";

export const store = configureStore({
  reducer: {
    counter: counterReducer,
  },
});
```

`configureStore` internally Redux DevTools aur middleware (jaise redux-thunk) ko automatically set kar deta hai, isliye manually configure karne ki zarurat nahi padti.

4. Provider se App ko Wrap Karna

Store banane ke baad, poori App ko `Provider` se wrap karna hota hai taaki har component store access kar sake.

```
// main.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import { Provider } from "react-redux";
import { store } from "./app/store";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

5. createSlice - Slice Banana

Redux Toolkit mein `createSlice` sabse important function hai. Isme hum ek jagah **initial state**, **reducers** (jo actions bhi generate kar dete hain automatically) define karte hain.

```
// features/counter/counterSlice.js
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  value: 0,
```

```
};

const counterSlice = createSlice({
  name: "counter",
  initialState,
  reducers: {
    increment: (state) => {
      state.value += 1; // Directly mutate kar sakte hain, Immer handle karta hai
    },
    decrement: (state) => {
      state.value -= 1;
    },
    incrementByAmount: (state, action) => {
      state.value += action.payload;
    },
  },
});

export const { increment, decrement, incrementByAmount } = counterSlice.actions;
export default counterSlice.reducer;
```

Note: Normal Redux mein directly state mutate karna galat practice hai, lekin Redux Toolkit `Immer` library use karta hai jo “mutate jaisa dikhne wala” code likhne dete hain, but background mein immutable update hi hoti hai.

6. Component Mein Use Karna - useSelector aur useDispatch

State read karne ke liye `useSelector` aur action dispatch karne ke liye `useDispatch` use karte hain.

```
import { useSelector, useDispatch } from "react-redux";
import { increment, decrement, incrementByAmount } from "./counterSlice";

function Counter() {
  const count = useSelector((state) => state.counter.value);
  const dispatch = useDispatch();

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => dispatch(increment())}>+</button>
      <button onClick={() => dispatch(decrement())}>-</button>
      <button onClick={() => dispatch(incrementByAmount(5))}>+5</button>
    </div>
  );
}

export default Counter;
```

`useSelector` state ke andar se sirf wahi part select karta hai jo chahiye (yahan `state.counter.value`), poora state nahi dobara render hota jab tak related part change na ho.

7. Multiple Slices Combine Karna

Real project mein aksar multiple slices hoti hain (jaise `counter`, `auth`, `cart`). Sabko store mein combine karte hain:

```
// app/store.js
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../features/counter/counterSlice";
import authReducer from "../features/auth/authSlice";
import cartReducer from "../features/cart/cartSlice";

export const store = configureStore({
  reducer: {
    counter: counterReducer,
    auth: authReducer,
    cart: cartReducer,
  },
});
```

8. Async Logic - createAsyncThunk

Jab API call jaisa async kaam karna ho (jaise data fetch karna), tab `createAsyncThunk` use karte hain.

```
// features/users/userSlice.js
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";

export const fetchUsers = createAsyncThunk("users/fetchUsers", async () => {
  const response = await fetch("https://api.example.com/users");
  const data = await response.json();
  return data;
});

const userSlice = createSlice({
  name: "users",
  initialState: {
    list: [],
    status: "idle", // idle | loading | succeeded | failed
    error: null,
  },
  reducers: {},
  extraReducers: (builder) => {
    builder
```

```
      .addCase(fetchUsers.pending, (state) => {
        state.status = "loading";
      })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.status = "succeeded";
        state.list = action.payload;
      })
      .addCase(fetchUsers.rejected, (state, action) => {
        state.status = "failed";
        state.error = action.error.message;
      });
  },
});

export default userSlice.reducer;
```

`createAsyncThunk` automatically 3 action types generate karta hai: `pending`, `fulfilled`, aur `rejected`. In teeno ko `extraReducers` mein handle karte hain.

Component mein use karna:

```
import { useEffect } from "react";
import { useSelector, useDispatch } from "react-redux";
import { fetchUsers } from "./userSlice";

function UserList() {
  const dispatch = useDispatch();
  const { list, status, error } = useSelector((state) => state.users);

  useEffect(() => {
    if (status === "idle") {
      dispatch(fetchUsers());
    }
  }, [status, dispatch]);

  if (status === "loading") return <p>Loading...</p>;
  if (status === "failed") return <p>Error: {error}</p>;

  return (
    <ul>
      {list.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

9. RTK Query (Bonus - Data Fetching Made Easy)

RTK Query Redux Toolkit ka built-in data fetching aur caching solution hai, jo `createAsyncThunk` se bhi zyada simplify karta hai API calls ko.

```
// features/api/apiSlice.js
import { createApi, fetchBaseQuery } from "@reduxjs/toolkit/query/react";

export const apiSlice = createApi({
  reducerPath: "api",
  baseQuery: fetchBaseQuery({ baseUrl: "https://api.example.com" }),
  endpoints: (builder) => ({
    getUsers: builder.query({
      query: () => "/users",
    }),
  }),
});

export const { useGetUsersQuery } = apiSlice;
```

Store mein add karna:

```
import { configureStore } from "@reduxjs/toolkit";
import { apiSlice } from "../features/api/apiSlice";

export const store = configureStore({
  reducer: {
    [apiSlice.reducerPath]: apiSlice.reducer,
  },
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(apiSlice.middleware),
});
```

Component mein use karna bahut easy hai:

```
import { useGetUsersQuery } from "../features/api/apiSlice";

function UserList() {
  const { data: users, isLoading, isError } = useGetUsersQuery();

  if (isLoading) return <p>Loading...</p>;
  if (isError) return <p>Error fetching users</p>;

  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
```

```
}
```

RTK Query automatically caching, refetching aur loading states handle kar leta hai, isliye manual `useEffect` aur `useState` likhne ki zarurat nahi padti.

10. Redux DevTools

Redux Toolkit ke `configureStore` se Redux DevTools automatically enabled ho jaate hain. Chrome/Firefox mein "Redux DevTools" extension install karke aap:

- Har dispatch hui action dekh sakte hain
- State ka before/after diff dekh sakte hain
- Time-travel debugging kar sakte hain (purani state par wapas ja sakte hain)

11. Common Hooks/Functions Summary

Function/Hook	Kaam
`configureStore()`	Store banane ke liye
`createSlice()`	Reducer + actions ek saath banane ke liye
`createAsyncThunk()`	Async API calls handle karne ke liye
`useSelector()`	Store se state read karne ke liye
`useDispatch()`	Action dispatch karne ke liye
`createApi()` (RTK Query)	Data fetching aur caching ke liye

12. Common Mistakes (Errors jo aksar hoti hain)

1. **Provider se wrap na karna** - App ko `<Provider store={store}>` se wrap karna bhool jaana, jisse "could not find store" error aati hai.
2. **Slice reducer export na karna** - `export default sliceName.reducer` likhna bhool jaana.
3. **Async logic reducers mein likhna** - `reducers` object sirf synchronous updates ke liye hai; async ke liye `createAsyncThunk` aur `extraReducers` use karna chahiye.
4. **Direct state return karna aur mutate dono** - `createSlice` ke andar ya to state ko mutate karo (direct assignment), ya naya state return karo - dono ek saath mat karo.

13. Conclusion

Redux Toolkit ne Redux ke saath kaam karna bahut easy bana diya hai. `createSlice` se boilerplate kam hota hai, `createAsyncThunk` se async logic clean rehta hai, aur RTK Query se to data fetching almost automatic ho jati hai. Agar aap React mein global state management seekh rahe ho, to Redux Toolkit hi industry-standard approach hai practice karne ke liye.